

ANO

2026



UNINTER

ATIVIDADE PRÁTICA

COMPILADORES

Prof. Diógenes Cogo Furlan, Me.



INTRODUÇÃO

Olá a todos.

Sejam todos muito bem-vindos a esta nova jornada de estudos!

Esta avaliação foi especialmente elaborada para as disciplinas de Compiladores do Centro Universitário Internacional UNINTER, visando consolidar os conhecimentos teóricos adquiridos ao longo da rota de estudos.

Neste roteiro, você encontrará:

- ✓ As orientações gerais para a realização da atividade,
- ✓ Os critérios de correção,
- ✓ O trabalho que deverá ser resolvido,
- ✓ E todas as informações necessárias para que você possa demonstrar, na prática, o que aprendeu.

Desejamos a todos um excelente desenvolvimento dos estudos e da atividade prática.

Que seja um período produtivo e repleto de aprendizados!



SUMÁRIO

INTRODUÇÃO	2
ORIENTAÇÕES GERAIS	4
ESTRUTURA DO CADERNO DE RESPOSTAS	4
FORMATO DE ENTREGA	5
CUIDADOS IMPORTANTES	5
PASSO A PASSO PARA MONTAR SEU DOCUMENTO FINAL	6
DICAS FINAIS	6
CRITÉRIOS DE AVALIAÇÃO	7
ATIVIDADE PRÁTICA	8
ENUNCIADO	8
GRAMÁTICA	9
EXEMPLOS DE ENTRADA EM MINI-LISP (POSITIVOS)	10
EXEMPLOS DE ENTRADA E SAÍDA (NEGATIVOS)	11
EXEMPLOS DE SAÍDA (POSITIVOS)	12

ORIENTAÇÕES GERAIS

ESTRUTURA DO CADERNO DE RESPOSTAS

1. Capa

- Inclua seu **nome completo** e seu **RU**.
- Caso não conste o nome e o RU, a atividade poderá ser zerada por falta de identificação.

2. Exemplos negativos

- Para cada exemplo, siga a ordem:
- Programa em mini-LISP
- Captura de tela (Screenshot) da execução com a mensagem de erro

3. Exemplos positivos

- Para cada exemplo, siga a ordem:
- Programa em mini-LISP
- Captura de tela (Screenshot) da execução (lista de tokens/lexemas, tabela de símbolos, código MEPA)

4. Código-Fonte Completo

- Insira todo o código que você desenvolveu, desde a primeira até a última linha.
- O código deve estar indentado e organizado, facilitando a leitura.
- Adicione **comentários** no código para explicar, com suas palavras, o que cada parte ou trecho faz.

4. Captura de Tela (Screenshot)

- Após os exemplos, inclua uma imagem mostrando o terminal em execução, exibindo o resultado do seu programa para aquele exemplo.
- Essa imagem serve para comprovar que seu código foi executado corretamente e está mostrando a saída solicitada.

CUIDADO!

Em programação, não existem dois códigos exatamente iguais. Cada programador organiza seu código de uma forma diferente, declara variáveis com nomes diferentes, faz comentários diferentes, gera mensagens aos usuários distintas etc. Por este motivo, não serão aceitos dois algoritmos idênticos entre alunos (ou iguais à Internet). Caso o corretor observe respostas iguais, elas serão consideradas como PLÁGIO e será atribuída a NOTA ZERO na questão.

FORMATO DE ENTREGA

- ✓ Você deverá utilizar o **Caderno de Respostas**; este arquivo vai unir todo o conteúdo (capa, códigos e capturas de tela) em um único arquivo.
- ✓ A Atividade Prática, em formato DOCX ou PDF, deve ser enviado no AVA-Univirtus, no campo/icone disponibilizado para entrega de **Trabalhos**.
- ✓ O modelo do **Caderno de Respostas** (em Word) está disponível no AVA-Univirtus.
 - Baixe o modelo.
 - Preencha-o com suas soluções, seguindo a estrutura acima.
 - Exporte ou imprima em PDF para realizar o envio.
- ✓ Atenção: arquivos em formatos diferentes do DOCX ou PDF não serão corrigidos e a nota será zero.

CUIDADOS IMPORTANTES

1. Evite Plágio

- a. Cada código deve ter a sua “digital”: nomes de variáveis, estruturas de repetição, formatação, comentários, estilo de programação etc.
- b. Não serão aceitos dois códigos idênticos (entre alunos ou copiados da internet).
- c. Em caso de cópias, será atribuída nota zero por **plágio**.

2. Validação do Código

- a. Antes de inserir o código no Caderno de Respostas, teste-o no seu computador ou ambiente de programação para garantir que está compilando/executando sem erros.
- b. Corrija eventuais problemas de sintaxe ou lógica.

3. Sobre as Capturas de Tela

- a. Mostre o prompt/terminal rodando e exibindo a saída do seu programa para cada exemplo.
- b. Se quiser colocar exemplos adicionais, é bem-vindo, mas não esqueça de mostrar o caso mínimo que o professor pede.

PASSO A PASSO PARA MONTAR SEU DOCUMENTO FINAL

1. **Abra o modelo de Caderno de Respostas (Word)** no AVA-Univirtus.
2. **Preencha a capa** com seu nome e RU.
3. **Preencha os dados do trabalho**
 - a. Copie e cole **todo** o seu **código-fonte** no campo indicado.
 - b. Insira os **exemplos** utilizados nos campos indicados
 - c. Insira a **captura de tela** do terminal nos campos indicados.
 - d. (Opcional) Acrescente uma breve explicação depois do print, se necessário.
4. Ao concluir todos os exercícios, revise o documento.
 - a. Verifique se há possíveis erros de digitação e se todos os comentários estão claros.
 - b. Confirme se está tudo bem-organizado (código, exemplos, prints da execução).
5. **Exporte ou Salve em PDF.**
 - a. Vá em “Arquivo” → “Salvar como” ou “Exportar” → escolha “PDF”.
 - b. Verifique se o arquivo ficou correto (capa, códigos, imagens).
 - c. Somente caso não consiga gerar o PDF, será aceitado o arquivo no formato DOCX.
6. Faça o **envio** no ícone de “Trabalhos” do AVA-Univirtus.
 - a. Verifique novamente se você enviou o PDF certo e se o nome do arquivo está adequado.

DICAS FINAIS

- ✓ **Testes de Funcionamento:** sempre rode seu programa várias vezes, testando diferentes entradas (quando aplicável), para garantir que ele se comporta conforme exigido.
- ✓ **Prazo de Entrega:** fique atento(a) à data limite de envio no AVA. Entregas após o prazo podem não ser aceitas ou terão descontos na nota, conforme as regras da disciplina.

CRITÉRIOS DE AVALIAÇÃO

Item de Avaliação	Critério Detalhado	Nota Máxima
1. Infraestrutura Base (Lexer e Parser)	O compilador lê o arquivo .lisp, isola os tokens corretamente e gera a Árvore de Sintaxe Abstrata (AST) interna estruturada (sem quebras).	2,0
2. Geração de Código: Núcleo Básico	Tradução correta de tipos primitivos (int, float), operações aritméticas (+, -, *, /) com parênteses corretos em MEPA, e chamadas de funções nativas (ex: CRVL).	2,0
3. Estruturas de Controle e Fluxo	Suporte completo e funcional para as estruturas condicionais (if), laços de repetição (while), blocos agregadores (begin) e atribuição de variáveis (set).	2,0
4. Análise Semântica	Geração da Tabela de Símbolos para programas com variáveis	2,0
5. Tratamento de Erros e Robustez	O compilador não sofre <i>crash</i> com entradas inválidas. Ele identifica e aponta erros léxicos (caracteres inválidos) ou sintáticos (parênteses desalinhados) de forma clara.	2,0
Total Geral	Sucesso em todos os requisitos obrigatórios.	10,0

ATIVIDADE PRÁTICA

ENUNCIADO

O objetivo deste projeto é consolidar os conceitos teóricos de linguagens formais e compiladores por meio da construção prática de um tradutor para a linguagem **Mini-Lisp**. O Lisp é historicamente reconhecido por sua sintaxe minimalista baseada em expressões simbólicas (*S-expressions*), onde código e dados compartilham a mesma estrutura de listas aninhadas. Neste trabalho, os desenvolvedores projetarão e implementarão um compilador capaz de ler um código-fonte escrito em Mini-Lisp e traduzi-lo para código **P-Code (MEPA)** semanticamente equivalente e pronto para execução, explorando os desafios clássicos de mapeamento de escopo, análise sintática e geração de código.

As seguintes linguagens podem ser usadas na codificação:

- C++
- Java / C#
- Python

O compilador deverá possuir 4 módulos:

- **Analisador Léxico** (verificação textual)
- **Analisador Sintático** (verificação das regras gramaticais)
- **Analisador Semântico** (construção da Tabela de Símbolos)
- **Gerador de código intermediário** (saída do programa para MEPA)

Demonstração do Funcionamento:

- Execute o programa utilizando pelo menos 3 exemplos negativos, ou seja, cuja saída dá algum tipo de erro;
- Execute o programa utilizando 2 exemplos de programa positivos, ou seja, cuja saída é correta. Neste caso deve ser gerado 3 resultados:
 - Lista de tokens/lexemas
 - Tabela de Símbolos
 - Código MEPA
- Faça as capturas de tela do terminal que mostrem os resultados obtidos para anexar ao seu caderno de respostas.
- Exemplos de programas e suas respectivas saídas se encontram a seguir.
- Não é necessário que haja tracejados na saída, apenas as informações das tabelas.
- No seu programa, formule seus próprios exemplos para entrega.

GRAMÁTICA DO MINI-LISP

Para viabilizar a construção do compilador dentro do cronograma estipulado, utilizaremos o **Mini-Lisp**, um subconjunto estrito e simplificado do dialeto Lisp original.

A linguagem suportará tipos primitivos numéricos (inteiros e pontos flutuantes), operações aritméticas básicas e operadores relacionais de comparação. Além disso, o escopo inclui estruturas de controle de fluxo essenciais, como condicionais (if) e loops (while), além de mecanismos para atribuição dinâmica de variáveis (set). Quaisquer recursos, funções integradas ou tipos de dados não explicitados nesta especificação serão sumariamente considerados fora de escopo.

A sintaxe aceita pelo Mini-Lisp é governada por uma **Gramática Livre de Contexto (GLC)** definida formalmente na notação BNF (Backus-Naur Form) abaixo.

```
<program> ::= <expr>
<expr>    ::= <atom> | <list>

<atom>    ::= <number> | <symbol>
<number>  ::= <integer> | <float>
<symbol>  ::= <operator> | <keyword> | <identifier>

<list>    ::= "(" <expr_list> ")"
<expr_list> ::= <expr> <expr_list> | ε
```

Para entender como o parser transforma o texto em árvore, vamos quebrar cada regra:

- **<program>**: O ponto de entrada do compilador. No nosso caso, um programa é composto por uma expressão principal (que pode conter sub-expressões).
- **<expr> (Expressão)**: A unidade fundamental do Lisp. Tudo é uma expressão. Uma expressão ou é um valor único (<atom>) ou uma estrutura aninhada entre parênteses (<list>).
- **<list>**: É qualquer coisa que comece com (, contenha uma sequência de expressões e termine com).
- **<expr_list>**: Representa zero ou mais expressões dentro dos parênteses. O símbolo ϵ (épsilon) significa "vazio", permitindo listas vazias como ().
- **<atom> (Átomo)**: Os elementos indivisíveis da linguagem (números e símbolos).

Para os terminais mais baixos da árvore (que o Analisador Léxico resolve via Expressões Regulares), nós temos:

- **<number>**: Inteiros (como 10) ou números de ponto flutuante (como 3.14).
- **<operator>**: Os caracteres mágicos +, -, *, /, %, >, <, ==, !=, <=, >=.
- **<keyword>**: Palavras reservadas pelo compilador (no nosso caso "if", "while", "begin", "set", "print").
- **<identifier>**: Nomes de funções ou variáveis (como "i", "x", "resultado").

EXEMPLOS DE ENTRADA EM MINI-LISP (POSITIVOS)

1) Print básico

```
(print 10)
```

2) Operações básicas

```
(begin
  (print (+ 10 (* 3 5)))
  (print (- 100 (/ 20 2)))
)
```

3) Uso de variáveis

```
(begin
  (set a 10)
  (set b 20)
  (print (+ a b))
)
```

4) Uso de condicional

```
(begin
  (set idade 20)
  (if (>= idade 18)
    (print 1)
    (print 0))
)
```

5) Uso de repetição

```
(begin
  (set contador 1)
  (while (<= contador 10)
    (begin
      (print contador)
      (set contador (+ contador 1))
    )
  )
)
```

EXEMPLOS DE ENTRADA E SAÍDA (NEGATIVOS)

1) Com Erro Léxico: Caractere Inválido

```
(begin  
  (set valor 100)  
  (print (valor @ 2)))
```

Mensagem esperada: Erro Léxico: Caractere inválido '@' na linha 3.

2) Com Erro Sintático: Parêntese não fechado

```
(begin  
  (print (+ 1 2)  
)
```

Mensagem esperada: Erro Sintático na linha 2: Fim de arquivo inesperado. Esperava-se ')'.
(print (+ 1 2))

3) Com Erro Sintático: Estrutura do IF incompleta

```
(if (> x 10) (print 1))
```

Mensagem esperada: Erro Sintático: Estrutura 'if' malformada. Omitido argumento de 'else'.

4) Com Erro Semântico: Variável não existente

```
(print a)
```

Mensagem esperada: Erro Semântico: Variável 'a' não foi inicializada.

EXEMPLOS DE SAÍDA (POSITIVOS)

1) Print básico

```
(print 10)
```

Análise Léxica / Sintática

NUM	TOKEN	LEXEMA
===	=====	=====
1	LPAR	(
2	PRINT	print
3	INTEGER	10
4	RPAR	)

Tabela de Símbolos

- Vazia

Geração de Código Intermediário

```
INPP  
CRCT 10  
IMPR  
PARA
```

2) Operações básicas

```
(begin
  (print (+ 10 (* 3 5)))
  (print (- 100 (/ 20 2))))
```

Análise Léxica / Sintática

NUM	TOKEN	LEXEMA
===	=====	=====
1	LPAR	(
2	BEGIN	begin
3	LPAR	(
4	PRINT	print
5	LPAR	(
6	OP	+
7	INTEGER	10
8	LPAR	(
9	OP	*
10	INTEGER	3
11	INTEGER	5
12	RPAR	)
13	RPAR	)
14	RPAR	)
15	LPAR	(
16	PRINT	print
17	LPAR	(
18	OP	-
19	INTEGER	100
20	LPAR	(
21	OP	/
22	INTEGER	20
23	INTEGER	2
24	RPAR	)
25	RPAR	)
26	RPAR	)
27	RPAR	)

Tabela de Símbolos

- Vazia



Geração de Código Intermediário

```
INPP  
CRCT 10  
CRCT 3  
CRCT 5  
MULT  
SOMA  
IMPR  
CRCT 100  
CRCT 20  
CRCT 2  
DIVI  
SUBT  
IMPR  
PARA
```

3) Uso de variáveis

```
(begin
  (set a 10)
  (set b 20)
  (print (+ a b)))
```

Análise Léxica / Sintática

NUM	TOKEN	LEXEMA
===	=====	=====
1	LPAR	(
2	BEGIN	begin
3	LPAR	(
4	SET	set
5	ID	a
6	INTEGER	10
7	RPAR	)
8	LPAR	(
9	SET	set
10	ID	b
11	INTEGER	20
12	RPAR	)
13	LPAR	(
14	PRINT	print
15	LPAR	(
16	OP	+
17	ID	a
18	ID	b
19	RPAR	)
20	RPAR	)
21	RPAR	)

Tabela de Símbolos

Identificador	Endereço MEPA	Tipo Presumido	Escopo
a	0	Inteiro	Global
b	1	Inteiro	Global

Geração de Código Intermediário

```
INPP
CRCT 10
ARMZ 0      ; 0 é o endereço MEPA da variável a
CRCT 20
ARMZ 1      ; 1 é o endereço MEPA da variável b
CRVL 0
CRVL 1
SOMA
IMPR
PARA
```

4) Uso de condicional

```
(begin
  (set idade 20)
  (if (>= idade 18)
    (print 1)
    (print 0)))
```

Análise Léxica / Sintática

NUM	TOKEN	LEXEMA
===	=====	=====
1	LPAR	(
2	BEGIN	begin
3	LPAR	(
4	SET	set
5	ID	idade
6	INTEGER	20
7	RPAR	)
8	LPAR	(
9	IF	if
10	LPAR	(
11	OP	>=
12	ID	idade
13	INTEGER	18
14	RPAR	)
15	LPAR	(
16	PRINT	print
17	INTEGER	1
18	RPAR	)
19	LPAR	(
20	PRINT	print
21	INTEGER	0
22	RPAR	)
23	RPAR	)
24	RPAR	)

Tabela de Símbolos

Identificador	Endereço MEPA	Tipo Presumido	Escopo
idade	0	Inteiro	Global

Geração de Código Intermediário

```
INPP
CRCT 20
ARMZ 0      ; 0 é o endereço MEPA da variável idade
CRVL 0
CRCT 18
CMEG
DSVF R1
CRCT 1
IMPR
DSVS R2
R1: NADA
CRCT 0
IMPR
R2: NADA
PARA
```

5) Uso de repetição

```
(begin
  (set contador 1)
  (while (<= contador 10)
    (begin
      (print contador)
      (set contador (+ contador 1))))))
```

Análise Léxica / Sintática

NUM	TOKEN	LEXEMA
===	=====	=====
1	LPAR	(
2	BEGIN	begin
3	LPAR	(
4	SET	set
5	ID	contador
6	INTEGER	1
7	RPAR	)
8	LPAR	(
9	WHILE	while
10	LPAR	(
11	OP	<=
12	ID	contador
13	INTEGER	10
14	RPAR	)
15	LPAR	(
16	BEGIN	begin
17	LPAR	(
18	PRINT	print
19	ID	contador
20	RPAR	)
21	LPAR	(
22	SET	set
23	ID	contador
24	LPAR	(
25	OP	+
26	ID	contador
27	INTEGER	1
28	RPAR	)
29	RPAR	)
30	RPAR	)
31	RPAR	)
32	RPAR	)

Tabela de Símbolos

Identificador	Endereço MEPA	Tipo Presumido	Escopo
contador	0	Inteiro	Global

Geração de Código Intermediário

```
INPP
  CRCT 1
  ARMZ 0      ; 0 é o endereço MEPA da variável contador
R1: NADA
  CRVL 0
  CRCT 10
  CMME
  DSVF R2
  CRVL 0
  IMPR
  CRVL 0
  CRCT 1
  SOMA
  ARMZ 0
  DSVS R1
R2: NADA
  PARA
```